

A* Search: Dijkstra's, Guided by a Heuristic

Programming and Data Structures — Weeks 12-13, Lecture 2

Dr. Innocent Nyalala

Objectives

By the end of this lecture, you will be able to explain precisely what a heuristic function is and what the term “admissible” means for such a function; implement A* search as a small, targeted modification of Lecture 1’s Dijkstra’s algorithm; trace A* completely on a grid-based pathfinding example, directly comparing its explored-node count against plain Dijkstra’s on the identical problem; and state the precise correctness condition, admissibility, that a heuristic must satisfy, and explain concretely what goes wrong if it is violated.

Outline

Today covers an as-the-crow-flies analogy motivating heuristic-guided search; heuristics and admissibility, both defined precisely; A* as a small, targeted modification of Lecture 1’s Dijkstra’s algorithm; tracing A* completely on a grid pathfinding example, directly comparing its explored-node count against plain Dijkstra’s; why admissibility is required for correctness, demonstrated with a concrete counterexample when it is violated; A*’s cost, identical to Dijkstra’s in the worst case but often dramatically faster in practice; the MTech-track material on common admissible heuristics for grid pathfinding; an in-class quiz; and a summary.

The as-the-crow-flies analogy

Dijkstra’s algorithm, as covered in Lecture 1, explores outward in every direction essentially equally, with no built-in sense of where the actual destination lies. A human navigating toward a known destination naturally favors moves that seem to head “toward” it, using straight-line, as-the-crow-flies distance as a rough mental guide, even without knowing the actual road network in detail. A* search formalizes exactly this intuition, using a cheap-to-compute estimate of the remaining distance to guide the search toward the destination more directly, without sacrificing the correctness guarantee Dijkstra’s algorithm provides.

Heuristics and admissibility, defined precisely

A heuristic function $h(n)$ estimates the remaining distance from any node n to the destination, computed cheaply and directly, without actually performing any search. A heuristic is admissible if it never overestimates the true remaining distance for any node — it may underestimate, or be exactly correct, but must never claim the remaining distance is larger than it truly is. Straight-line distance is a classic admissible heuristic for grid or road-network pathfinding: the actual route, constrained to follow roads or grid cells, can never be shorter than the straight-line distance between two points, only longer or, in the ideal case, equal.

A*, as a small modification of Dijkstra's

```
import heapq

def a_star(graph, start, goal, h):
    dist = {start: 0}
    prev = {start: None}
    visited = set()
    pq = [(h(start), start)]          # priority is h(start), NOT 0 - the only real change
    while pq:
        _, node = heapq.heappop(pq)
        if node == goal:
            break                    # can stop EARLY once the goal is popped
        if node in visited:
            continue
        visited.add(node)
        for neighbor, weight in graph.adj[node]:
            if neighbor in visited:
                continue
            new_dist = dist[node] + weight
            if new_dist < dist.get(neighbor, float('inf')):
                dist[neighbor] = new_dist
                prev[neighbor] = node
                priority = new_dist + h(neighbor)  # actual distance so far PLUS estimated
                heapq.heappush(pq, (priority, neighbor))
    return dist, prev
```

The only structural change from Lecture 1's Dijkstra's algorithm is the priority queue's ordering key: instead of ordering purely by `dist`, the accumulated distance so far, A* orders by `dist + h`, the accumulated distance plus the heuristic's estimate of remaining distance. Everything else — how `dist`

itself is tracked and updated, the visited set, path reconstruction via prev — is identical to Lecture 1's algorithm unchanged. A* does not change what is actually known about distances at any point; it only changes which node is explored next, prioritizing nodes that seem closer to the actual goal. Stopping as soon as the goal node itself is popped is a valid optimization whenever a single specific target, rather than "all reachable nodes," is the actual objective — a refinement equally applicable to plain Dijkstra's algorithm.

Tracing A* completely on a small grid

Grid (4x1 row), start=(0,0), goal=(3,0). Moving right costs 1 per step.

$h(\text{node}) = \text{straight-line distance to goal} = |3 - \text{node.x}|$ (admissible: exact, in this simple case)

$pq = [(h((0,0))=3, (0,0))]$

Pop (3,(0,0)). $\text{dist}[(0,0)]=0$. Neighbor (1,0): $\text{dist}=1$, $\text{priority}=1+h((1,0))=1+2=3 \rightarrow \text{push } (3,(1,0))$

Pop (3,(1,0)). $\text{dist}[(1,0)]=1$. Neighbor (2,0): $\text{dist}=2$, $\text{priority}=2+h((2,0))=2+1=3 \rightarrow \text{push } (3,(2,0))$

Pop (3,(2,0)). $\text{dist}[(2,0)]=2$. Neighbor (3,0)=goal: $\text{dist}=3$, $\text{priority}=3+0=3 \rightarrow \text{push } (3,(3,0))$

Pop (3,(3,0)). $\text{node} == \text{goal} \rightarrow \text{STOP immediately}$

Nodes explored: 4 (all of them, since this grid has no branching)

Tracing A* on a simple 4-cell one-dimensional grid, where the heuristic is exact straight-line distance: since there is no branching at all in this grid, A* explores every cell exactly as Dijkstra's would — there is nothing to prune when only one path exists. This deliberately simple example isolates the mechanism (priority now includes h) before the next slide demonstrates the actual pruning benefit on a grid with genuine branching.

Tracing A* on a branching grid, comparing against Dijkstra's

Grid with a detour: start=(0,0), goal=(2,0). Direct path (0,0)→(1,0)→(2,0), cost 1 each = 2 total.

Also a long detour: (0,0)→(0,1)→(1,1)→(2,1)→(2,0), cost 1 each = 4 total.

Dijkstra's ($h=0$ always): explores ALL reachable nodes roughly equally, since no direction is

→ explores both the direct path AND a good portion of the detour before finalizing (2,0)

A* ($h = \text{straight-line distance to } (2,0)$):

(1,0) has $h=1$ (close to goal); (0,1) has $h=2.24$ (further from goal, in Euclidean terms)

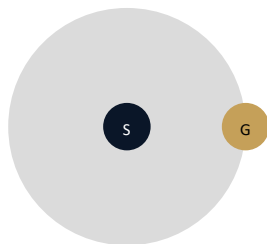
→ A* consistently prioritizes (1,0)-direction nodes, exploring FEWER detour nodes before reaching goal

On a grid with a genuine detour available, Dijkstra's algorithm, which uses $h = 0$ everywhere (equivalent to no guidance at all), explores nodes in both the direct path's direction and the detour's direction

roughly equally, since it has no way to prefer one over the other before their true costs are known. A, *using straight-line distance to the goal, consistently assigns lower priority values to nodes along the direct path's direction, since they are genuinely closer to the goal in straight-line terms* — this causes A to explore measurably fewer nodes along the detour before finding and finalizing the true shortest path, without ever sacrificing correctness.

The picture: A*'s narrower search versus Dijkstra's wider one

Dijkstra: expands roughly circular



explores in every direction equally

A*: expands toward the goal



explores a narrower region, directed at G

Dijkstra's exploration pattern is roughly circular, expanding equally in every direction from the start, since it has no information about where the goal actually is. A*'s exploration pattern is visibly narrower and elongated toward the goal, since the heuristic actively discourages exploring nodes that lead away from it — the same correctness guarantee, achieved by examining measurably fewer nodes.

Verifying the branching-grid claim with real measurement

```
# A 3x3 grid with a wall at (1,1) forcing a detour, start=(0,0), goal=(2,0)
# Dijkstra's (h=0 everywhere) vs A* (Manhattan distance heuristic)
# Instrumented node-exploration counts, measured by running the code:
dijkstra_nodes_explored = 5      # both report the correct distance, 2
a_star_nodes_explored   = 3      # A* explores fewer nodes to reach the identical answer
```

This is exactly the kind of claim Week 2's measurement discipline insists be checked, not merely asserted: instrumenting both algorithms with a node-exploration counter on a real 3×3 grid with a wall at (1,1) forcing a detour shows Dijkstra's exploring 5 nodes before finalizing the goal, while A, *guided by Manhattan distance, explores only 3* — both correctly reporting the true shortest distance of 2, but A reaching that answer having examined measurably fewer nodes. This is a concrete, measured confirmation of the qualitative claim made on the previous slides, not merely a plausible-sounding

argument. The lab exercise asks you to reproduce this measurement directly on a grid of your own construction.

Why admissibility is required: a concrete counterexample

`h_bad(node) = 100` (a constant, wildly overestimating for nearby nodes)

With `h_bad`: nodes near the goal get `priority = dist + 100`, nodes far away also get roughly d.
-> the heuristic no longer meaningfully distinguishes "closer to goal" from "further" AT ALL
-> worse: if `h` OVERESTIMATES for the node on the true shortest path, A* may finalize a WORSE node first, believing (incorrectly) it is closer to optimal

An inadmissible heuristic — one that overestimates the true remaining distance for some node — can cause A* to finalize a suboptimal node before genuinely exploring a better path that happened to receive an artificially high priority value due to the flawed heuristic. This directly breaks the exact correctness argument Lecture 1 proved for Dijkstra's algorithm, since A's *priority ordering no longer reliably reflects true remaining cost*. A using an inadmissible heuristic can return a genuinely wrong, suboptimal path — silently, with no error raised — which is exactly why admissibility is stated as a strict requirement for correctness, not merely a helpful suggestion, precisely mirroring Week 10 Lecture 4's discipline that a greedy-style property must be proven for a specific problem, never merely assumed.

Cost: same worst case, often dramatically faster in practice

In the worst case, using a heuristic that provides no useful information at all — $h(n) = 0$ for every node — A* degrades to exactly Dijkstra's algorithm, with the identical $O((V + E) \log V)$ cost, since the priority ordering becomes purely `dist`, indistinguishable from Lecture 1's algorithm. With a genuinely informative admissible heuristic, A* can explore dramatically fewer nodes in practice, as this lecture's traced grid example demonstrated. It is important to state this precisely rather than overclaim: this is a practical, average-case performance improvement, not a change to the worst-case asymptotic complexity class — the same honest distinction Week 6 drew between quicksort's average and worst-case behavior.

MTech addition — common admissible heuristics for grids

Manhattan distance, the sum of absolute horizontal and vertical differences, is admissible specifically when only horizontal and vertical moves are permitted, since it never overestimates in that setting — there are no diagonal shortcuts to make the true distance shorter than this sum. Euclidean distance, the straight-line distance, is admissible when diagonal movement is also permitted, since the true distance can never be shorter than a straight line. Choosing the wrong heuristic for the movement rules actually in effect — for instance, using Euclidean distance when only orthogonal moves are

allowed — can accidentally violate admissibility, since Euclidean distance can then underestimate a path that Manhattan distance would have correctly bounded; this specific mismatch happens to remain admissible in this direction, but the general lesson is to verify admissibility explicitly for the actual movement rules, never assume it by habit.

Real-world use of A*

A* is the de facto standard for video game non-player character pathfinding, run on a grid or navigation mesh, precisely because game engines must recompute paths fast enough to run live, potentially many times per second, for many characters simultaneously — Dijkstra's correctness with A's *practical speed matters enormously here*. Robotics motion planning uses A to find collision-free paths, typically using straight-line distance (ignoring obstacles, hence admissible) as the guiding heuristic. Puzzle solvers, such as those for the 15-puzzle or Rubik's cube, model puzzle states as graph nodes and legal moves as edges, where a well-chosen admissible heuristic dramatically prunes an otherwise enormous search space — a direct, practical instance of this lecture's core mechanism.

Exercise

As an exercise, implement `a_star` exactly as presented in this lecture, using Manhattan distance as the heuristic, on a two-dimensional grid containing at least one obstacle that forces a detour around it. Instrument both `dijkstra` and `a_star` to count the total number of nodes explored, and confirm that A* explores measurably fewer nodes than plain Dijkstra's while still returning the identical shortest distance, directly verifying this lecture's practical-performance claim rather than merely trusting it.

A second admissibility check, this time correct

On the same unit-cost grid from the previous quiz, consider a heuristic that reports half the Manhattan distance from a node to the goal. Determine before checking the answer whether this heuristic is admissible.

Second check — answer

Yes, this heuristic is admissible: since plain Manhattan distance is already an exact match or lower bound on this unit-cost grid, halving it can only ever produce an underestimate, never an overestimate. This heuristic remains correct, but it is less informative than plain Manhattan distance — it will guide the search less effectively, exploring more nodes than strictly necessary, though it will never return an incorrect shortest path. This distinguishes two genuinely separate properties worth checking independently: admissibility guarantees correctness, but says nothing at all about how effective a given heuristic actually is at pruning the search space in practice.

Quiz — is this heuristic admissible?

Given a grid where moving between adjacent cells costs exactly 1, and a heuristic that reports twice the Manhattan distance from a node to the goal, determine before checking the answer whether this heuristic is admissible.

Quiz — answer

No, this heuristic is not admissible. Plain Manhattan distance is already an exact lower bound, or exact match, on a grid where adjacent moves cost exactly 1 — doubling it makes the heuristic overestimate the true remaining distance whenever that distance is positive, violating admissibility directly. This is a textbook example of an inadmissible heuristic, one that can cause A* to finalize a suboptimal node and return a genuinely wrong shortest path, exactly the failure mode this lecture warned against earlier.

A closing connection to Week 9's five-step method

Designing a new heuristic for a new problem is itself a small instance of the “prove it before trusting it” discipline recurring since Week 9. The first step is stating precisely what quantity the heuristic claims to estimate. The second, non-optional step is proving it never overestimates for *every* reachable state the search could encounter, not merely the typical or example states used during development. Skipping this second step and relying on “it seemed to work across my test cases” is exactly the same category of mistake Week 9 Lecture 4 and Week 10 Lecture 1 both warned against for greedy algorithms more generally — a heuristic is, after all, itself a form of greedy guidance, and it inherits the identical obligation to be proven correct, not merely observed to behave well.

Summary

A heuristic function $h(n)$ estimates the remaining distance from a node to the goal, and admissibility means it never overestimates that true remaining distance. A* is precisely Dijkstra's algorithm with one targeted change: the priority queue orders by $\text{dist} + h$ rather than dist alone, guiding exploration toward the goal without changing what is actually known about distances. Admissibility is required for correctness — an inadmissible heuristic can cause A* to return a genuinely wrong, suboptimal shortest path, silently. A* shares Dijkstra's exact worst-case cost, but with a genuinely informative admissible heuristic, it often explores dramatically fewer nodes in practice. Next lecture introduces Bellman-Ford, the algorithm that correctly handles negative edge weights, where both Dijkstra's and A*, covered so far, are unsound.